Impact of Quantization and Structured Pruning on Battery SOH Prediction Models across Architectures

Jonas Schulze[b], Nils Strassenburg[b], Aleinikau Pavel[a], Florian Rzepka[a,*]

[a]Technical University of Berlin, Chair of Electrical Energy Storage Technology, Einsteinufer 11, Berlin, 10587, Berlin, Germany
[b]Hasso-Plattner-Institute, Chair of Data Engineering Systems, Prof.-Dr.-Helmert-Str. 2-3, Potsdam, 14482, Potsdam, Germany

Abstract

- Core objective: investigate how pruning, quantization, and knowledge distillation affect State of Health (SOH) prediction quality and embedded deployment cost across Long Short-Term Memory (LSTM), Gated Recurrent Unit (GRU), Convolutional Neural Network (CNN), and Temporal Convolutional Network (TCN) architectures.
- Study principle: architectures are trained under a harmonized protocol first, then optimized under controlled compression settings to enable fair cross-architecture comparison.
- Evaluation focus: Mean Average Error (MAE)/Root Mean Squared Error (RMSE) on SOH prediction and embedded KPIs (flash, RAM, latency, energy).
- Expected outcome: architecture-specific design guidance for resource-constrained BMS implementations.

Keywords: Battery SOH prediction, Structured pruning, Quantization, Knowledge distillation, Embedded AI, LSTM, GRU, CNN, TCN, BMS

1. Introduction

The growing demand for low-carbon energy systems and electrified transportation has increased the importance of reliable electrochemical energy storage [1, 2]. Lithium-ion batteries are widely used because they combine high energy and power density with long cycle life and a low self-discharge rate [3, 4, 5]. To support safe and efficient operation, a battery management system measures voltage, current, and temperature and estimates internal states and health indicators that cannot be measured continuously during operation [6, 7]. The capacity-based SOH used in this work is defined as

$$\mathrm{SOH}(t) = \frac{Q_{\mathrm{current}}(t)}{Q_{\mathrm{initial}}}, \tag{1}$$

where $Q_{\mathrm{current}}(t)$ is the currently available capacity and Q_{initial} is the initially measured available capacity of the same cell. A value of one corresponds to 100 % SOH. The value generally decreases as degradation progresses toward a defined end-of-life criterion [8, 9].

Accurate SOH estimation remains challenging [10, 11], particularly when the estimator must operate under the computational constraints of embedded battery management hardware [12]. Available approaches

*Corresponding author
Email address: florian.rzepka@tu-berlin.de (Florian Rzepka)

Preprint submitted to Engineering Applications of Artificial Intelligence

differ substantially in their computational requirements and dependence on prior knowledge. Physics-based methods use detailed electrochemical descriptions and can require computationally intensive partial differential equations [13, 14]. Equivalent circuit models reduce this complexity but still depend on experimental parameter identification and model assumptions [7, 15].

Data-driven methods have therefore received increasing attention in SOH estimation research [16]. Rather than explicitly representing the internal electrochemical processes, they learn a mapping between measured operating data and battery health from historical observations [10]. Recurrent architectures such as LSTMs and GRUs, as well as convolutional architectures such as CNNs and TCNs, have all been applied to this task [16, 17]. Their deployment on resource-constrained microcontrollers can nevertheless require optimization through techniques such as structured pruning and quantization [18]. Tiny machine learning demonstrates that neural battery-state estimators can be executed at the edge [19], but results obtained for different architectures are often based on different datasets, splits, input representations, and hardware platforms.

This study investigates whether the effects of structured pruning and INT8 quantization depend on the temporal architecture used for SOH prediction. One preselected representative of each of four architecture families, LSTM, GRU, CNN, and TCN, is evaluated relative to its own uncompressed FP32 baseline. Before optimization, the baselines are compared using MAE, RMSE, and parameter-derived FP32 weight size. After structured pruning and INT8 quantization, the variants are assessed in terms of prediction error and serialized model size. Their deployment behavior is evaluated separately through measured inference latency on an STM32H753ZI-class microcontroller and USB-side power and energy per prediction for the complete board.

All representatives use the same cell-level data partition and target definition from the Lithium Iron Phosphate DoE Cycle Aging Dataset [20]. The cells were subjected to long-term cycling under controlled laboratory conditions with periodic capacity check-ups and hybrid pulse power characterization tests. By evaluating changes relative to each architecture-specific baseline, the study distinguishes baseline accuracy from architecture-dependent compression behavior and derives deployment guidance for embedded battery management systems. The complete analysis is made publicly available [1].

2. Related Work and Research Gap

Among data-driven SOH estimators, recurrent and convolutional networks represent two established approaches to temporal feature extraction. LSTMs are frequently used because their recurrent state can capture long-term aging patterns. Nguyen Van et al. [21], for example, developed a two-stage LSTM for joint SOH and internal-resistance estimation that outperformed a feedforward neural-network reference. Xu et al. [22] combined an LSTM with data-characteristic processing and spatio-temporal attention to improve estimation accuracy and robustness. Ungurean et al. [23] reported that a GRU required approximately 25 % fewer parameters than a compared LSTM, while retaining a similar prediction-error range. Vaishnani et al. [24] likewise found that a GRU achieved the lowest RMSE of 0.017 among the recurrent configurations evaluated in their study.

Convolutional architectures offer a different mechanism because temporal features can be evaluated in parallel without recurrent state propagation. Yang et al. [17] proposed an attention-enhanced TCN that achieved an average RMSE of 2.10 % and outperformed the reference methods considered in that work. Bi et al. [25] reported an approximately 3 % improvement of the TCN over the LSTM in terms of average maximum relative error under their accelerated-aging conditions. Aliberti et al. [26] compared an LSTM, a GRU, a one-dimensional CNN, and a hybrid CNN-LSTM. In that study, the one-dimensional CNN yielded the lowest estimation error and prediction variance.

These results demonstrate the suitability of several temporal architectures for SOH estimation, but they do not establish a generally superior architecture. The reported rankings were obtained using different battery datasets, input features, cell partitions, target definitions, and error metrics. Their numerical results are therefore informative within each study but cannot be used as a controlled cross-architecture benchmark.

[1] GitHub repository: https://github.com/hpides/batmm

Fewer studies connect SOH prediction quality with deployment costs after model compression. Kim et al. [27] developed an edge-compatible SOH framework using knowledge distillation, structured pruning, and dynamic quantization. The resulting models achieved more than 99 % compression relative to the authors' reference and were validated on a Raspberry Pi 4B. This demonstrates the potential of combined compression, but the study optimizes one framework rather than comparing how multiple recurrent and convolutional architectures respond under the same data and microcontroller conditions.

Consequently, evidence remains limited on whether the relative benefits and prediction-quality effects of structured pruning and INT8 quantization transfer consistently between representative recurrent and convolutional SOH estimators. The relevant gap is a controlled comparison that uses a shared cell-level split and prediction target, evaluates each optimized model relative to its own FP32 baseline, and applies a common embedded benchmark on the same microcontroller platform. Addressing this gap makes it possible to separate differences in baseline prediction quality from differences in compression sensitivity and to relate both to serialized model size, measured inference latency, and board-level electrical behavior.

3. Methodology

3.1. Comparison Objective and Benchmark Design

The comparison is designed to determine whether the effects of model optimization transfer consistently across neural architectures or depend on the mechanism used to represent temporal battery behavior. For this purpose, strong uncompressed baseline models are first established for the LSTM, GRU, CNN, and TCN families. Structured pruning and quantization are then applied under controlled conditions. The benchmark therefore examines not only whether these methods reduce model size and embedded execution cost, but also whether the resulting loss of SOH prediction accuracy follows a comparable pattern across architectures. This makes it possible to identify cases in which an expected compression benefit generalizes across model families and cases in which a particular architecture is more sensitive to parameter removal or reduced numerical precision.

The comparison is organized around the change from each architecture-specific FP32 baseline rather than around baseline accuracy alone. Prediction quality is evaluated using MAE and RMSE, while deployment cost is characterized through model footprint and the embedded metrics model size, latency, and energy. Model sizes may differ where this is required to establish competitive baseline quality. However, the data partition, target definition, preprocessing, error metrics, optimization settings, and benchmark protocol are held fixed wherever applicable. The resulting accuracy–resource trade-offs provide the basis for selecting optimization strategies according to both architecture and deployment constraints.

3.2. Data, Target, and Split Strategy

The experiments use the publicly available LFP DoE Cycle Aging Dataset recorded with 15 commercial 1.8-Ah, 3.2-V LFP 18650 cells at a controlled ambient temperature of 25 °C [28, 20]. The mixed full- and fractional-factorial design varies charge rate, discharge rate, and depth of discharge. Its factorial core covers C_{chg} = 0.5–0.9 C, C_{dis} = 1.0–2.5 C, and a depth of discharge of 45–65 %, with additional boundary conditions extending the investigated operating range. Periodic check-ups comprising capacity tests, open-circuit-voltage sweeps, and hybrid pulse-power characterization track degradation between the aging loops. Stationary-storage load profiles complement the controlled cycling conditions with application-oriented power trajectories [29]. The raw 1-Hz records contain voltage, current, terminal temperature, and auxiliary measurements.

The prediction target is the capacity-based state of health, defined as the ratio between the measured available capacity and the initially measured available capacity of the same cell. Capacity references are obtained from the periodic check-ups and interpolated along each aging trajectory. Figure 1 shows the resulting reference trajectories and their pronounced spread across operating conditions. Local increases between successive check-ups reflect variability in the measured available capacity and do not indicate a reversal of long-term aging. For model training, one SOH target is assigned to each hourly feature vector.

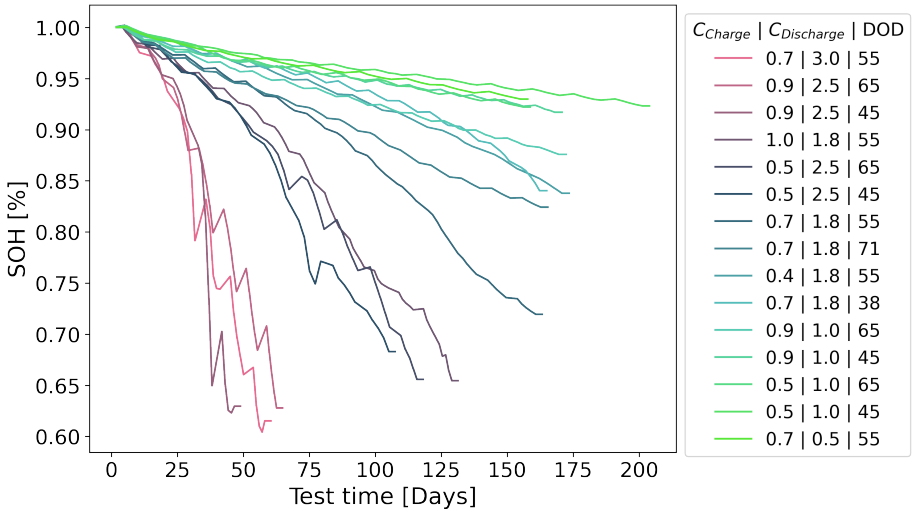


Figure 1: Capacity-based SOH reference trajectories of the 15 LFP cells over the aging campaign. Colors denote the investigated combinations of charge rate, discharge rate, and depth of discharge. The trajectory spread provides the common data basis for training and comparing the four neural architectures.

The partition is performed by complete cells to prevent samples from one cell from appearing in more than one subset. Cells C01, C03, C05, C07, C09, C13, C17, C19, C25, and C27 form the training set, C15 and C21 form the validation set, and C11, C23, and C29 are retained exclusively for testing. This cell-level split and the resulting hourly samples are held fixed across all four architecture families.

3.3. Architecture Portfolio and Baseline Selection

The portfolio comprises two recurrent architectures, an LSTM and a GRU, and two convolutional architectures, a CNN and a TCN. This selection enables compression to be compared across different mechanisms for temporal representation instead of within a single model family. All four models receive the same 20-channel input representation obtained from the hourly mean, standard deviation, minimum, and maximum of voltage, current, temperature, EFC, and cumulative charge. One SOH target is assigned to each hourly interval. Each model is causal and produces one SOH estimate at every time step.

The recurrent models use a learned feature projection followed by a unidirectional recurrent stack, layer normalization, residual MLP blocks, and an MLP output head. The selected LSTM contains two recurrent layers with a hidden size of 192 and three residual blocks. The GRU uses three recurrent layers with a hidden size of 160 and four residual blocks. The convolutional models process the same sequences using residual causal convolutions. The CNN contains three blocks with 128 channels, a kernel size of five, and dilation factors of 1, 2, and 4. The TCN extends this dilation pattern to 1, 2, 4, 8, and 16 across five blocks with 96 channels, thereby providing a wider temporal context without recurrent state propagation. Table 1 summarizes the selected uncompressed FP32 baselines.

The reference configuration of each architecture was identified through automated family-specific hyperparameter optimization using validation RMSE. The selected configuration was the best observed candidate within the predefined search space rather than a claim of a global optimum. After these reference configurations had been fixed, a targeted capacity sweep evaluated coordinated reductions and increases in the internal widths while retaining the remaining architecture and training settings. The investigated hidden or channel widths were 64–256 for the LSTM, 48–256 for the GRU, 64–224 for the CNN, and 48–144 for the TCN. This controlled post-selection analysis tests whether substantially smaller or larger models consistently improve generalization. The held-out test results were not used for model reselection.

Table 1: Architecture comparison for the SOH optimization study. Sequence lengths are given in hourly steps, and FP32 memory assumes four bytes per trainable parameter.

Family	Input steps	Architecture	Parameters	FP32 weights
LSTM	192	Feature projection, 2 recurrent layers ($H = 192$), 3 residual MLP blocks	895,937	3.418 MiB
GRU	168	Feature projection, 3 recurrent layers ($H = 160$), 4 residual MLP blocks	844,321	3.221 MiB
CNN	96	3 residual causal blocks ($C = 128$, $k = 5$, $d = 1/2/4$)	503,297	1.920 MiB
TCN	120	5 residual temporal blocks ($C = 96$, $k = 5$, $d = 1/2/4/8/16$)	439,841	1.678 MiB

Table 2: Training settings of the selected FP32 baseline models. The epoch entry gives the maximum number of training epochs, and λ_s denotes the weight of the temporal smoothness penalty.

Family	Batch size	Epochs	Learning rate	Weight decay	λ_s	Penalty
LSTM	128	60	9.28×10^{-5}	3.49×10^{-4}	0.0830	L1
GRU	128	60	2.81×10^{-4}	2.60×10^{-5}	0.0051	L2
CNN	48	50	3.23×10^{-4}	3.76×10^{-5}	0.0843	L1
TCN	64	60	1.97×10^{-4}	5.88×10^{-5}	0.0480	L1

3.4. Training and Evaluation Protocol

All architectures were trained with the same cell-level partitioning and random seed of 42. A robust scaler was fitted exclusively to the training cells and subsequently applied without refitting to the validation and test cells. The hourly feature vectors were arranged as overlapping causal sequences with a stride of one. Their architecture-specific lengths are listed in Table 1. Each model was trained in a sequence-to-sequence setting and therefore returned one SOH estimate for every hourly step. The first 8 steps were excluded from the loss for the recurrent models, compared with 12 steps for the CNN and 16 steps for the TCN, to reduce the influence of state initialization and incomplete convolutional context.

The training objective combined the mean squared prediction error with a temporal smoothness penalty,

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2 + \lambda_s \mathcal{R}(\Delta\hat{y}), \qquad \Delta\hat{y}_t = \hat{y}_t - \hat{y}_{t-1}, \tag{2}$$

where $\mathcal{R}$ denotes the mean absolute step change for the LSTM, CNN, and TCN and the mean squared step change for the GRU. The smoothness term discourages implausible hour-to-hour fluctuations without imposing a fixed degradation trajectory. AdamW was used for parameter optimization, followed by a cosine-annealing schedule with warm restarts. Gradient clipping and early stopping limited unstable updates and unnecessary training. Table 2 reports the architecture-specific settings retained after the hyperparameter search.

Validation was performed every three epochs. The checkpoint with the lowest validation RMSE was retained, with an early-stopping patience of 12 epochs for the LSTM, GRU, and TCN and 10 epochs for the CNN. The held-out test cells were used only after model and capacity selection. Prediction quality was quantified by

$$\mathrm{MAE} = \frac{1}{N} \sum_{i=1}^{N} |\hat{y}_i - y_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2}. \tag{3}$$

During baseline evaluation, each of the three held-out test cells was processed as one continuous causal trajectory. The recurrent states of the LSTM and GRU were propagated throughout each trajectory, while the CNN and TCN retained the input history required by their causal receptive fields across successive processing blocks. The metrics were first calculated separately for each test cell and then macro-averaged

so that every cell contributed equally to the reported baseline result. The subsequent compression and embedded benchmarks retain their architecture-specific execution modes. The CNN and TCN process non-overlapping model input windows, whereas the recurrent states of the LSTM and GRU are propagated sample by sample. All reported metrics use unsmoothed model outputs and the dimensionless 0–1 SOH scale. Accordingly, an MAE or RMSE of 0.01 corresponds to one SOH percentage point on the 0–100% scale.

3.5. Model Optimization Methods

Pruning. Pruning, in the scope of our work, is the process of removing weights from a model. Typically, the model components that contribute the least to a model's performance are candidates for removal. Pruning can be classified as either unstructured or structured. In unstructured pruning, we pinpoint the exact, individual weights we seek to remove. As the name suggests, the resulting model loses its structure; thus, to keep the matrices in an integral form, the selected weights are simply set to zero. We achieve high sparsity with minimal loss in accuracy, but the irregular matrices do not run faster on standard hardware that does not support sparse execution. In structured pruning, entire weight clusters, such as matrix rows/columns, filters, and channels, are removed. This physically shrinks the model architecture, making it faster and more efficient on typical hardware, but usually comes with a sharper drop in accuracy than unstructured pruning.

We incorporate structured pruning by using the TorchPruner library [30].

Distillation. Knowledge distillation transfers information from a larger, pretrained teacher model to a smaller student model. During training, the student learns not only from the ground-truth SOH values but also from the teacher's predictions, which provide additional information about the function learned by the larger model. This helps the student retain prediction accuracy despite its reduced capacity. Once training is complete, only the student is deployed, so distillation introduces no additional inference latency overhead. In our experiments, as the teacher for a compressed student of the same architecture, we evaluate both the uncompressed model and the previously pruned model in the iterative pruning setting.

Quantization. Quantization refers to reducing the precision of model parameters, usually the weights. Typical models are stored in FP32 (32-bit floating point), meaning each parameter uses 32 bits to represent it. When quantized, these parameters would then only be saved with e.g., 16 (FP16), 8 (INT8) or 4 (INT4) bits. This directly results in smaller file sizes. If the underlying hardware supports the quantized representation, these models can be executed significantly faster at minimal accuracy loss. Quantization can be paired with pruning to leverage both techniques. In this case, the model is usually pruned first, and the resulting, sparse model is quantized afterward.

For our experiments, we quantize to 8-bit precision, which compresses our models from FP32 to INT8.

- Figure 6 (planned, position in this subsection): optimization pipeline diagram showing baseline model, individual optimization branches, and combined branch.

3.6. Embedded Benchmark Protocol

For our benchmarks, we use a Python script to iterate over the folder of compressed models generated by the methods in Subsection 3.5. We benchmark all model variants with the same target platform and firmware toolchain. The script creates an isolated project in STM32CubeIDE (version 2.1.1) using STM32CubeMX (version 6.17) and STEdgeAI (version 4.0) for the machine-learning backend, then flashes it to our H753ZI microcontroller. The microcontroller is also connected to a USB AVHzY CT-3 power meter, which measures power draw over 100-millisecond intervals. An overview of our benchmarking setup is shown in Figure 2.

After building the project, we sequentially evaluate the test set of battery cells on the microcontroller using the input-window lengths specified in Table 1. Inference is performed with a batch size of 1 because deployment is stateful and operates sample-by-sample in online BMS setups. For the CNN and TCN, the complete input windows are sufficiently small to be processed in a single inference pass. The recurrent networks, LSTM and GRU, exceeded the microcontroller's available runtime memory, primarily because of the memory required for intermediate activations. We execute one step at a time and explicitly preserve their recurrent states, i.e., hidden and cell states for LSTM, and only the hidden states for GRU. Provided that

the states are initialized and propagated consistently, this step-wise execution is mathematically equivalent to processing the complete input sequence in a single inference call. To ensure fairness, we track results only after 192 and 168 steps, respectively.

Metrics. We track the MAE, its bias, and the maximum absolute error between the compressed model's battery SOH prediction and the ground truth. We measure the average inference latency of a single prediction and the size of the compressed models on disk. With our power meter, we measure the average voltage, current, and power consumption during inference.

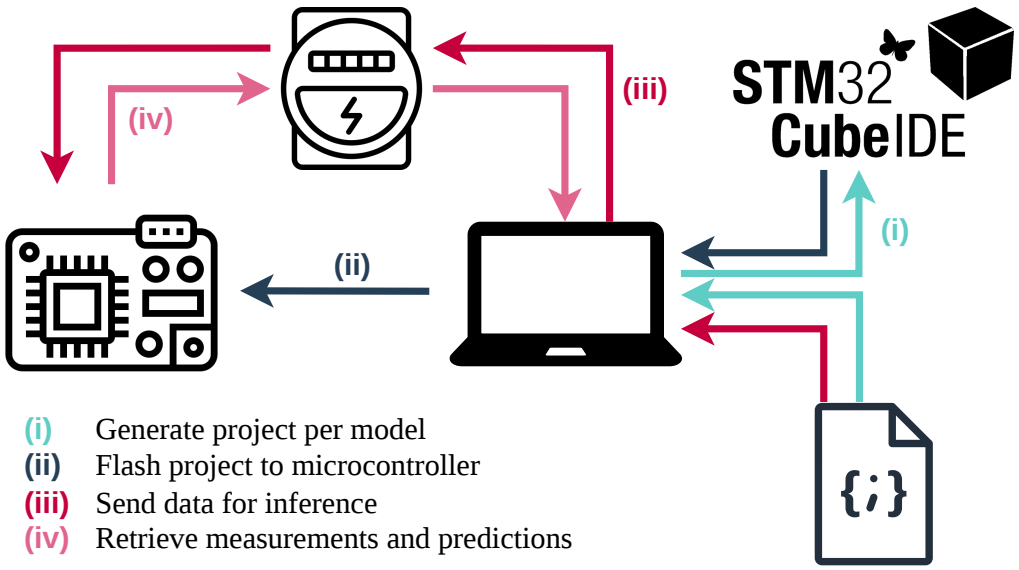


Figure 2: Setup for running benchmarks on the microcontroller.

4. Results

4.1. Baseline Model Comparison

The uncompressed FP32 models were first compared independently of any optimization. Figure 3(a) reports cell-macro errors across the three held-out test cells using the 0–1 SOH scale defined in Section 3.4. The MAE is 0.0172 for the CNN, 0.0146 for the GRU, 0.0167 for the LSTM, and 0.0152 for the TCN. The corresponding RMSE values are 0.0224, 0.0180, 0.0199, and 0.0196. The GRU therefore provides the lowest cell-macro error for both metrics in this baseline comparison, while the TCN follows closely with the smallest model footprint.

The parameter footprints differ more clearly than the prediction errors. As shown in Figure 3(b), the TCN and CNN require 1.678 and 1.920 MiB of FP32 weights, respectively, compared with 3.221 MiB for the GRU and 3.418 MiB for the LSTM. The largest baseline is therefore approximately twice the size of the smallest one despite their similar prediction accuracy. These differences provide distinct starting points for evaluating the architecture-dependent effects of compression in the following sections.

Figure 4 complements the aggregated errors with a continuous-context visualization for one held-out test cell. All selected baseline models reproduce the long-term SOH decline and the principal local variations. Their deviations from the reference nevertheless differ over time, which explains why the aggregate ranking cannot be inferred from a single operating interval. This view is illustrative, while Figure 3(a) remains the quantitative comparison across the complete test set.

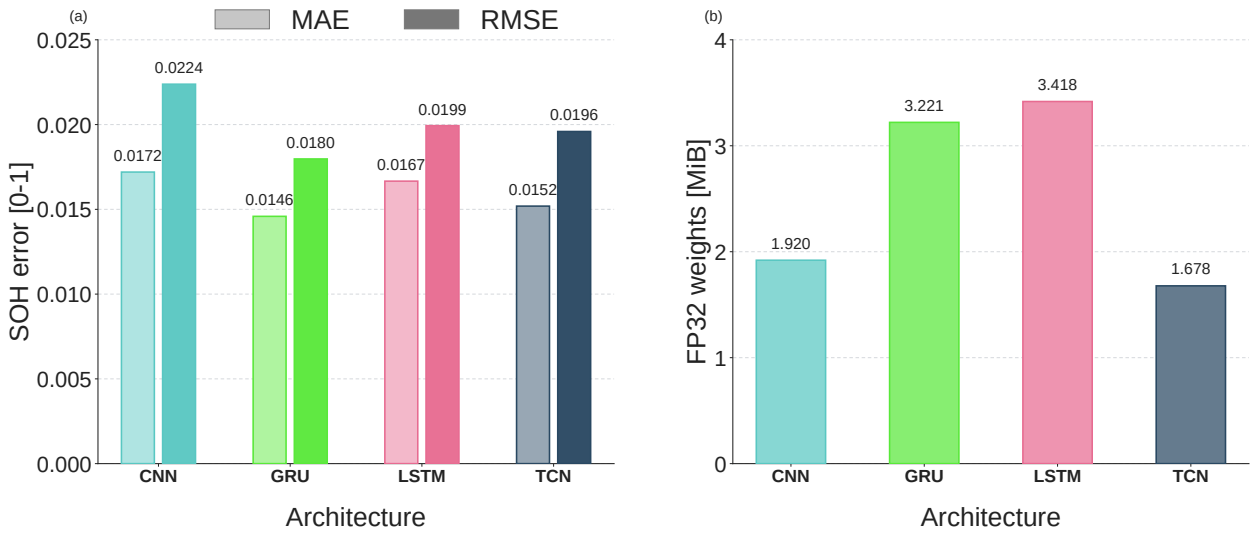


Figure 3: Comparison of the uncompressed FP32 baseline models. (a) Cell-macro MAE and RMSE across the three held-out test cells using continuous causal inference and the dimensionless 0–1 SOH scale. Each test cell contributes equally to the reported values, and an error of 0.01 corresponds to one SOH percentage point. (b) Parameter-only FP32 memory calculated with four bytes per trainable parameter.

The targeted capacity sweep provides an additional post-selection sensitivity check around the fixed reference configuration of each architecture. Figure 5 compares the cell-macro MAE of coordinated width reductions and increases across all three held-out test cells. The relationship between parameter count and prediction error is non-monotonic across all four families. Increasing or decreasing capacity therefore does not consistently improve generalization, while some compact variants remain competitive with substantially larger networks. The sweep is not used as a second model-selection step.

Only MAE is shown in Figure 5 to keep the four-family comparison legible. In each panel, base denotes the fixed reference model used in Figure 3. The labels s1–s4 and l1–l3 identify successively smaller and larger variants relative to that model. Each point is the unweighted mean of the three cell-specific errors, consistent with the cell-macro aggregation used for the baseline comparison.

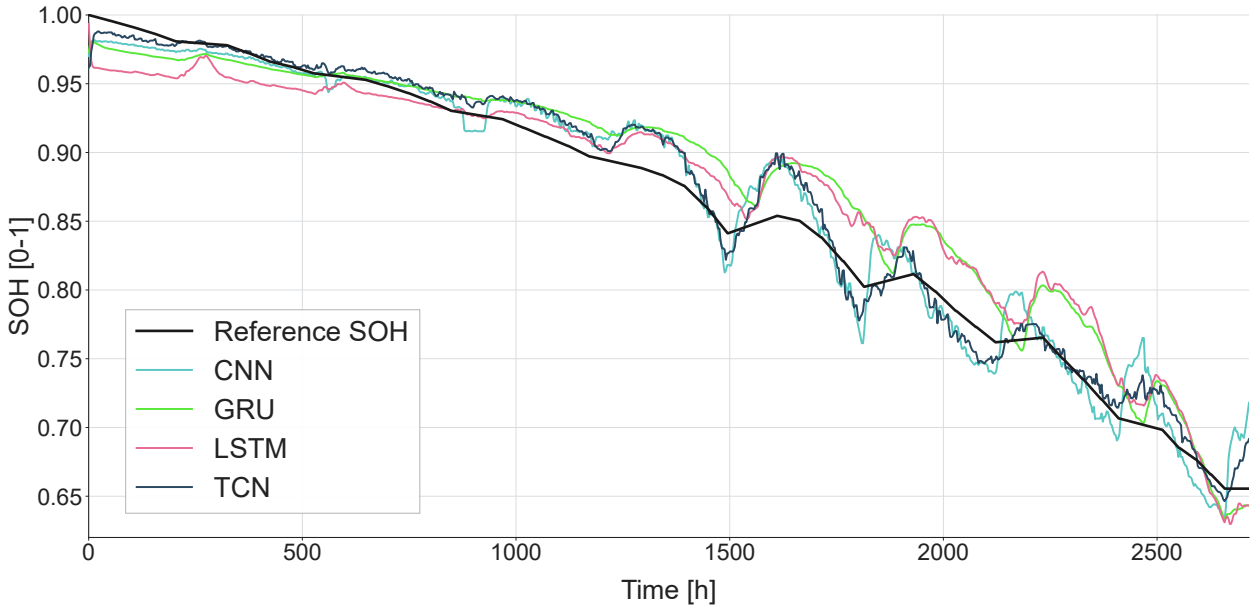
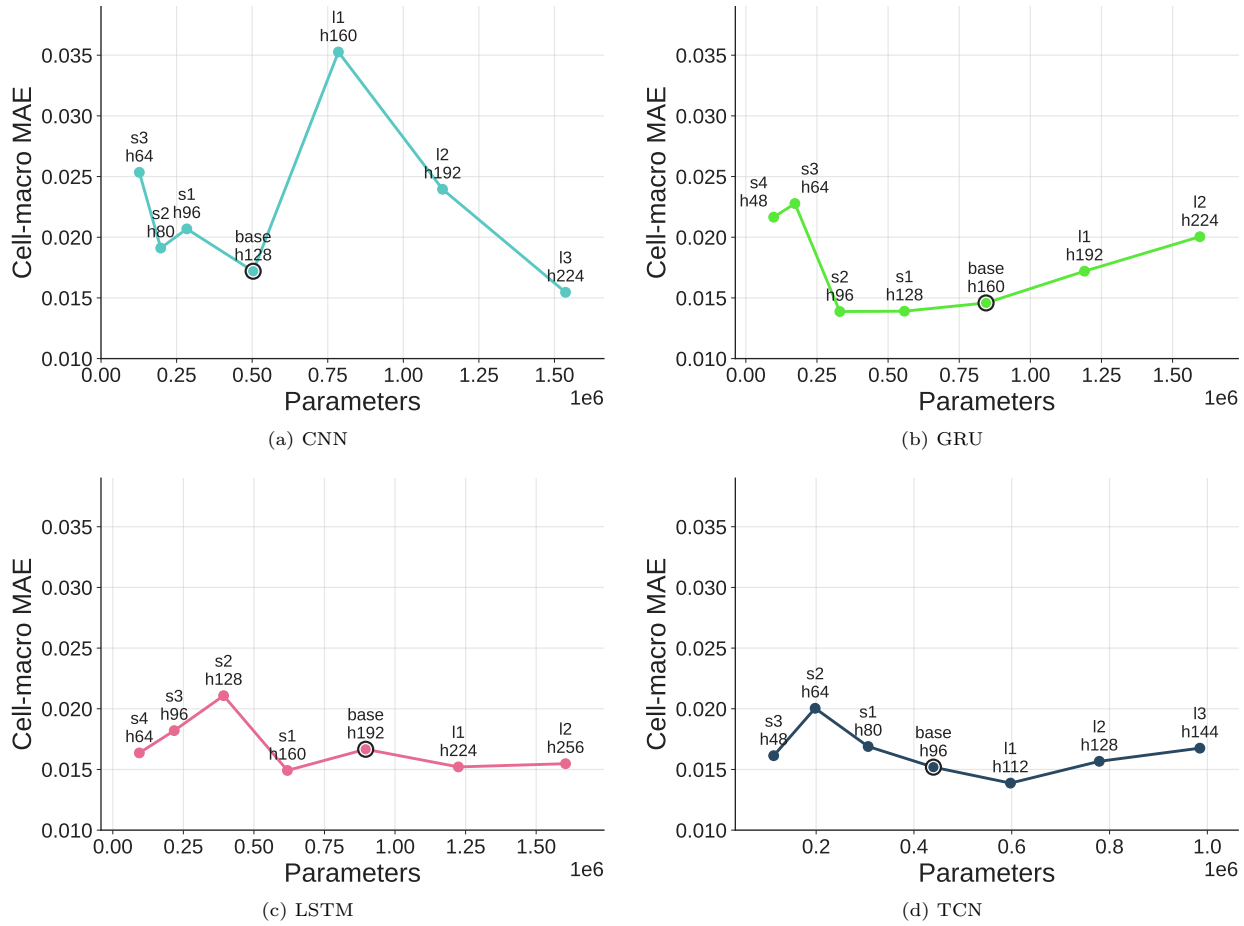


Figure 4: SOH trajectories of the selected uncompressed FP32 baseline models for one held-out test cell over 2716 hourly samples under continuous causal inference. The black line denotes the reference SOH, and the colored lines denote the predictions of the four architecture families. For legibility, the displayed prediction traces use a centered seven-hour median and every third plotting point. Smoothing and point reduction are applied only to this illustrative trajectory. The cell-macro errors in Figure 3(a) are calculated from unsmoothed model outputs across all three test cells.



(a) CNN

(b) GRU

(c) LSTM

(d) TCN

Figure 5: Post-selection capacity sensitivity across the three held-out test cells. The label base denotes the fixed reference model used in Figure 3, while s1–s4 and l1–l3 denote successively smaller and larger variants as applicable. Each point reports the cell-macro MAE on the normalized 0–1 SOH scale, where 0.01 corresponds to one SOH percentage point. Identical MAE-axis limits are used in all four panels.

4.2. Results

In this subsection, we present the evaluation results for our benchmarks across accuracy, latency, model size, and electrical power.

Accuracy. Figure 6 compares prediction errors across all four architectures, showing the mean, maximum, and bias of the errors.

For the CNN, we examine that the uncompressed base model does not fit in the limited memory of the microcontroller. While both quantized and non-quantized models slightly increase MAE, with up to 60% of parameters remaining, they both improve as pruning continues, with the quantized model performing best on average at 30% of parameters remaining; only the RMSE is marginally larger. Even with only 15% remaining, the model performs as well as the completely unpruned model.

The bias remains steady across all levels of pruning, indicating that the model over- and underpredicts equally.

TCN exhibits irregular behavior, incorporating multiple severe spikes and slopes in its mean prediction error. The quantized model slightly outperforms the base model at moderate pruning levels, both on average and, especially, in terms of a smaller RMSE. TCN is also the only model for which the bias is not symmetrical, as in early pruning stages, when the model collapses, it tends to underpredict. Experiments have shown that the model predicted a constant value, resulting in an inherent impact on the error.

Both recurrent architectures fit only in quantized form on the microcontroller. Since the LSTM and GRU blocks themselves cannot be structurally pruned, we prune only the neural network part of each, which accounts for roughly half of the parameter count.

For the LSTM, mean and squared error trends are relatively equal. Similar to the CNN, the error is worse in the early pruning stages but peaks when a quarter of the parameters are removed. The model is overpredicting the error by a small margin.

GRU shows a similar trend; however, the maximum error steadily increases here. Compared to LSTM, pruning up to 60% of the parameters still yields a well-performing model.

For all models, the INT8 version achieves the lowest error at relatively moderate pruning levels, while aggressive pruning falls within the same range as the base model, even for unquantized models.

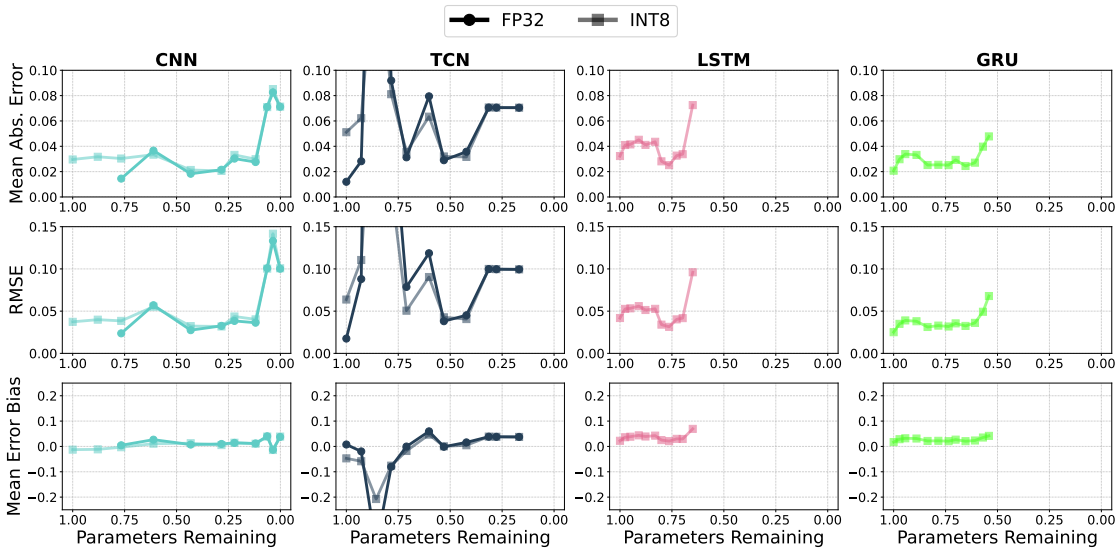


Figure 6: Prediction errors. The less opaque data points denote quantized models. For LSTM and GRU, only the quantized models fit the microcontroller.

Inference Latency. Figure 7 shows the inference latency for predicting a single battery value across all four architectures.

For CNNs, we see a linear decline in latency for both unquantized and quantized models. The quantized model shows a constant speed difference of roughly 850 ms relative to the unquantized model, resulting in the fastest time for a single prediction at 320 ms.

TCN is faster than the CNN at every pruning degree with a similar symmetric trend. Even though it needs comparatively more input steps, the TCN is smaller and has fewer parameters. But because the TCN has fewer prunable parameters than the TCN, it caps out earlier and has a minimum latency of only 340 ms. The unquantized TCN model is closer in latency to its quantized variant than the CNN model.

LSTM and GRU both exhibit near-constant speeds of 280 ms and 195 ms, respectively. Pruning the surrounding multi-layer perceptron yields a negligible latency reduction of less than 5 ms. This shows that the recurrent layers we do not prune are the dominant factor in inference speed.

While pruning recurrent architectures does not yield latency improvements, they are faster than convolutional architectures at every degree of pruning.

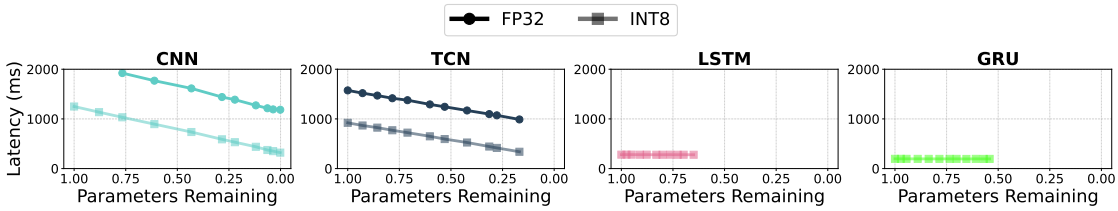


Figure 7: Inference latency of a single prediction for all architectures.

Model Size. Figure 8 shows the effect of pruning on the model size on disk.

The CNN is too large to fit on the microcontroller in its uncompressed form. While the unquantized architecture is always more memory-intensive than its quantized counterpart, pruning is, as expected, more effective because it removes 32 bits per parameter rather than 8. The same bit-width difference explains the consistent fourfold gap between the two model sizes across all pruning degrees.

The same constant decrease in size can be examined for the TCN and the recurrent architectures within their respective pruning capabilities. While the non-recurrent parts of the LSTM and GRU networks do not affect latency, they still account for roughly half of the architecture's size.

Overall, both compression techniques consistently reduce size across all model types.

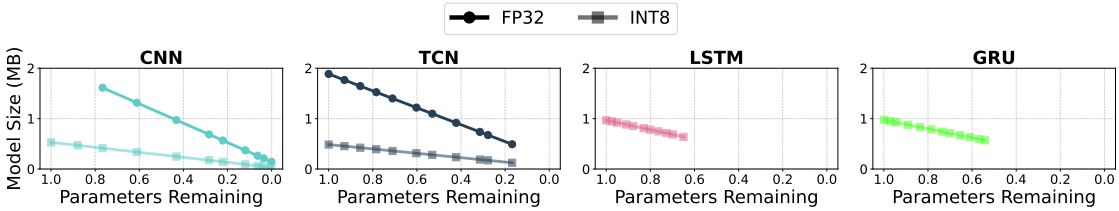


Figure 8: Model sizes on disk for all architectures.

Electrical Power. Figure 9 reports the average USB-input voltage, current, power, and energy cost per prediction of the complete microcontroller board during inference.

For all architectures, the supply voltage remains nearly constant at 5.08–5.10 V across compression levels, with quantized models being marginally lower than unquantized ones. Consequently, the observed power variations, according to $P = U \cdot I$, are primarily caused by changes in current draw.

For the CNN, pruning generally reduces the average current and power, most clearly for the INT8 variants, whose power decreases from approximately 1.50 W to 1.34 W. The TCN shows a similar but less regular tendency, including an isolated drop for the FP32 model at approximately 40% of parameters remaining.

In contrast, the quantized LSTM and GRU remain nearly constant at 1.28–1.31 W. This behavior is consistent with the latency results because only the surrounding feed-forward layers are pruned, while the recurrent layers, which dominate execution time, remain unchanged.

The INT8 convolutional models occasionally exhibit higher average power than their FP32 counterparts; however, this does not necessarily imply greater energy consumption, as the quantized models execute considerably faster, as seen in the energy-related plots. Consequently, the energy-per-prediction curves largely follow the latency trends shown in Figure 7, because variations in inference time dominate the comparatively small variations in average power. Thus, the reduced latency of an INT8 model can offset its higher instantaneous power draw, since energy per prediction equals the product of average power and inference time.

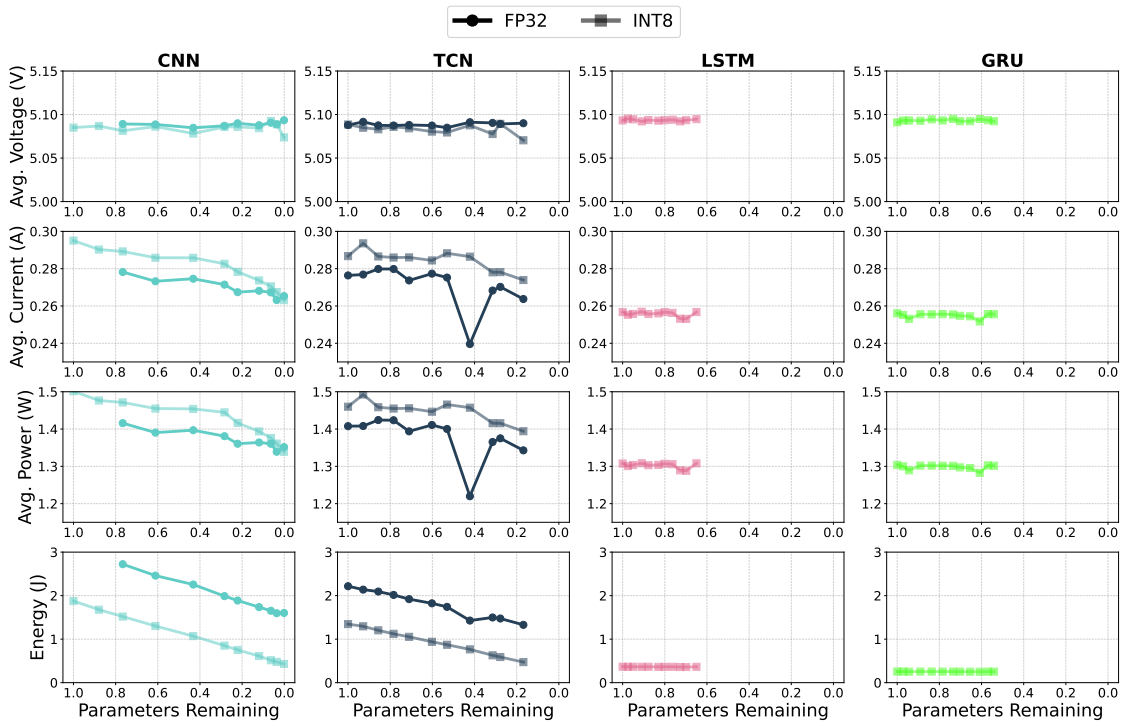


Figure 9: Average USB-side supply voltage, current, input power, and energy per prediction for all architectures.

4.3. Distillation

TODO (experiments still running).

In Figure 10, we see the impact of distillation during pruning.

Inter-Model Comparison. In Figure 11, we see a Pareto comparison between all architectures and compression techniques. Better models have lower MAE, size, and inference time, as indicated by the bottom-left corner in all Pareto plots.

We examine the interaction between model size and prediction error in Figure 11a. For almost all scenarios, the CNN dominates every other model. The quantized CNN are the smallest and maintain a good MAE. The moderately pruned, unquantized CNN are the most accurate and are only beaten by the uncompressed TCN, which, in return, is also the largest model.

Figure 11b compares all architectures based on latency and MAE. The fastest models are GRUs, which have a slightly lower latency than the LSTMs. The unpruned but quantized GRU is the most accurate while being essentially as fast as the other models in its architecture family. One of the more heavily pruned CNN

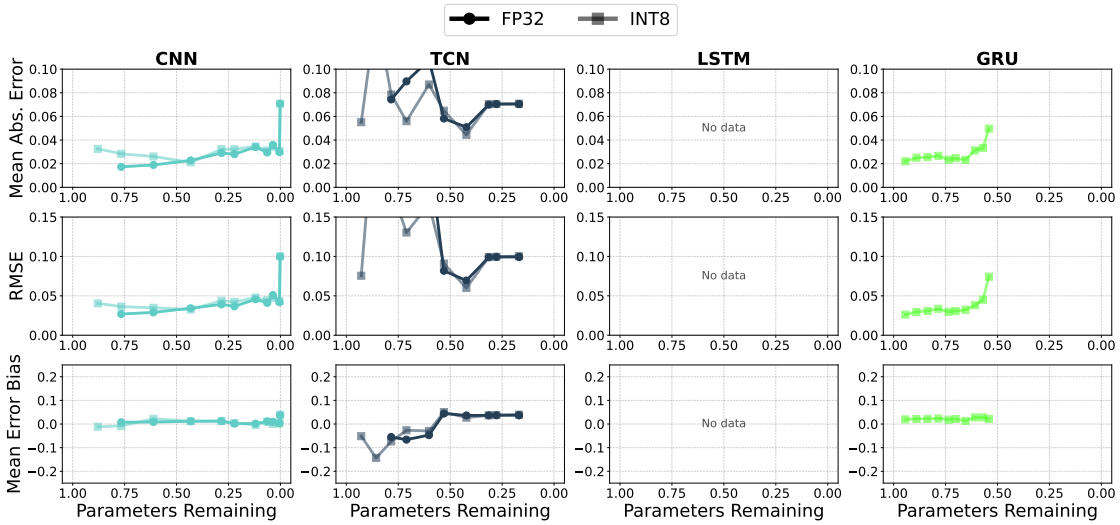


Figure 10: Performance of distilled models with the uncompressed model as teacher.

achieves a marginally smaller MAE, while being noticeably slower. Again, the most accurate model is the uncompressed TCN, but at the cost of significant latency.

Overall, every architecture besides the LSTMs is represented in the Pareto front. If the focus lies on small models, CNNs are the best. If latency is the highest priority, choose a GRU; if accuracy is most important, the uncompressed TCN is best.

In Figure 12, we show the SOH trajectories of two compressed CNN models from the Pareto front in FP32 and INT8 precision. We include the ground truth of the test cell and the uncompressed CNN model for comparison.

At extreme pruning degrees, the model predicts a constant value for healthy battery cells, as seen in Figure 12a. This is most likely because early-life feature differences are small or ambiguous. Below a battery SOH of 0.93, the model follows a trajectory similar to that of the uncompressed CNN, even though the base model has more than eightfold the parameters. In Figure 12b, we examine that quantizing this model afterward, resulting in a model smaller than 100 kilobytes, still predicts the SOH as well.

When the proportion of remaining parameters is increased to just under 30%, the model produces accurate predictions during the early stages, as shown in Figures 12c and 12d.

We see that it is possible to heavily compress and still receive capable models.

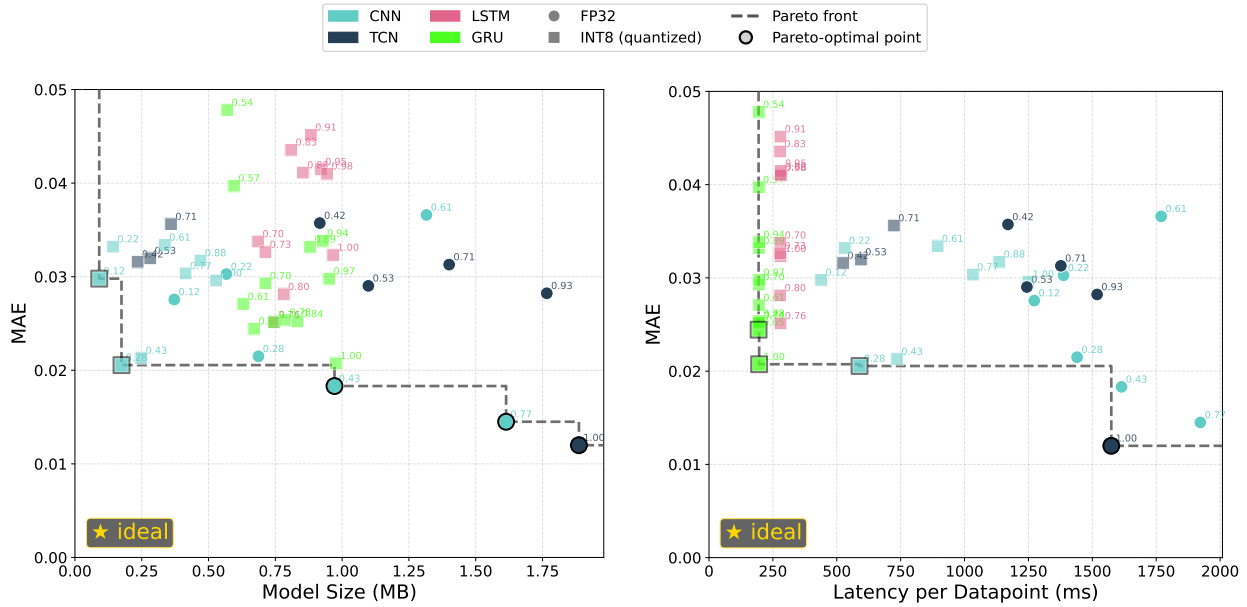
- Figure 12 (planned, position in this subsection): final decision map for architecture and optimization selection.

5. Discussion

5.1. Baseline Architecture Comparison (Flo, EET)

The baseline comparison shows that architecture choice affects resource demand more clearly than SOH prediction accuracy. Although all four models operate within a narrow MAE and RMSE range, their FP32 parameter footprints differ by approximately a factor of two. The GRU achieves the lowest cell-macro MAE and RMSE, whereas the TCN provides the smallest baseline footprint. Consequently, no architecture is uniformly superior across prediction error and storage requirements. The appropriate baseline depends on whether the application prioritizes estimation accuracy or memory demand.

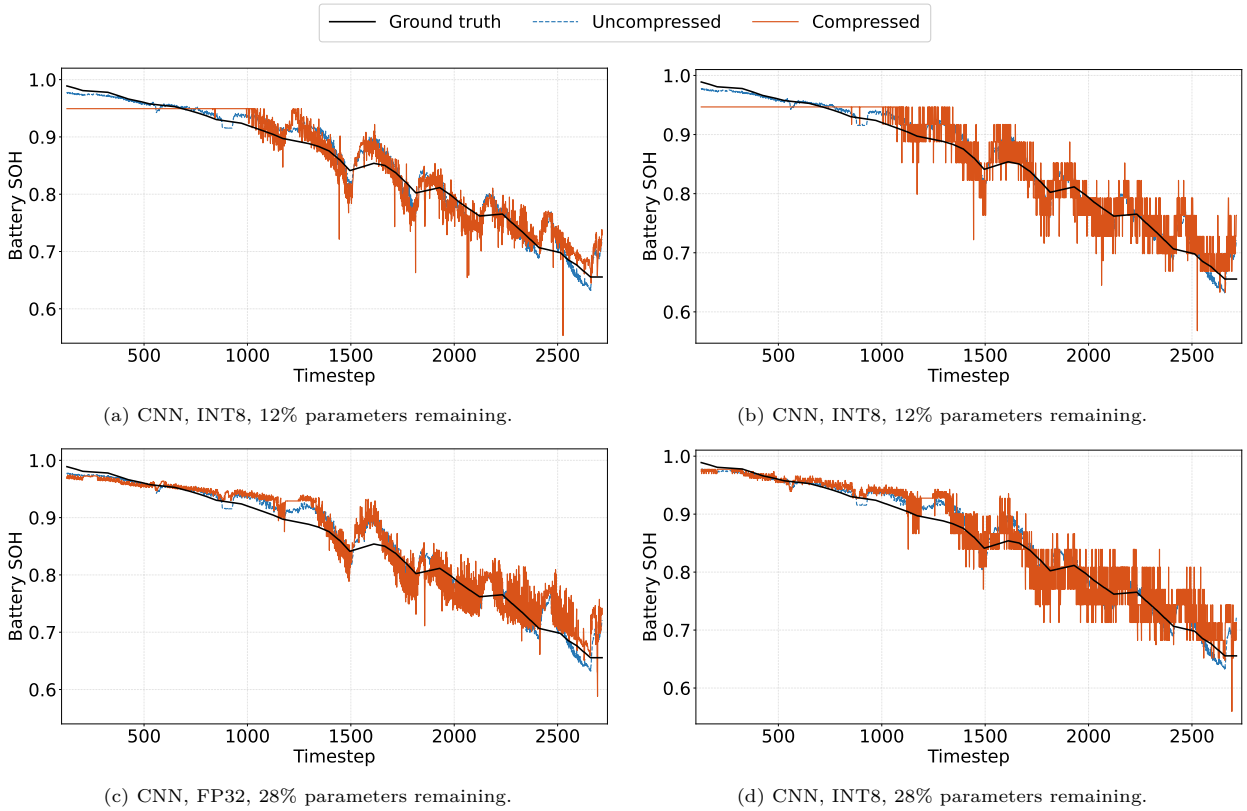
The post-selection capacity sweeps further show that parameter count does not produce a monotonic accuracy improvement. Neither increasing nor decreasing model capacity consistently reduces the three-cell



(a) Model size versus MAE.

(b) Latency versus MAE.

Figure 11: Pareto fronts for all evaluated architectures. The number next to each data point indicates the percentage of parameters remaining.



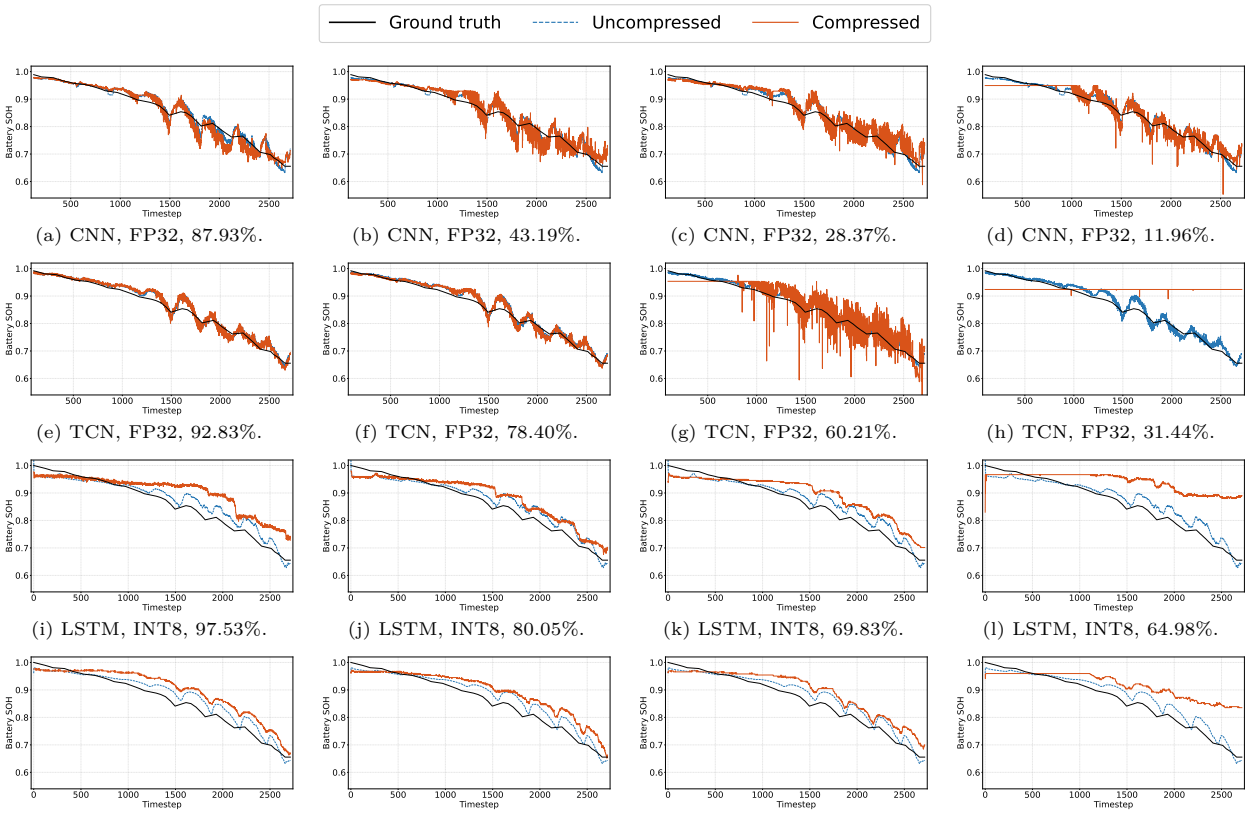
(a) CNN, INT8, 12% parameters remaining.

(b) CNN, INT8, 12% parameters remaining.

(c) CNN, FP32, 28% parameters remaining.

(d) CNN, INT8, 28% parameters remaining.

Figure 12: SOH trajectories of a heavily pruned CNN (12% and 28% of parameters remaining), both unquantized and quantized, on test cell C11.



(a) CNN, FP32, 87.93%. (b) CNN, FP32, 43.19%. (c) CNN, FP32, 28.37%. (d) CNN, FP32, 11.96%.

(e) TCN, FP32, 92.83%. (f) TCN, FP32, 78.40%. (g) TCN, FP32, 60.21%. (h) TCN, FP32, 31.44%.

(i) LSTM, INT8, 97.53%. (j) LSTM, INT8, 80.05%. (k) LSTM, INT8, 69.83%. (l) LSTM, INT8, 64.98%.

(m) GRU, INT8, 94.40%. (n) GRU, INT8, 78.47%. (o) GRU, INT8, 60.85%. (p) GRU, INT8, 54.14%.

Figure 13: SOH trajectories for the CNN, TCN, LSTM, and GRU architectures on test cell C11. Percentages indicate the parameters remaining after pruning; models are ordered from the largest to the smallest parameter fraction within each row. INT8 versions are displayed for the recurrent networks because the FP32 versions do not fit on the microcontroller.

test error across architectures. Compact configurations can remain competitive, while larger variants may generalize less effectively despite their additional capacity. This result supports the use of architecture-specific hyperparameter optimization instead of selecting the largest feasible network by default. It also separates the fixed reference-model selection from the subsequent compression comparison. The trajectory comparison provides a complementary single-cell view because all selected models reproduce the long-term degradation trend, while their local deviations occur at different points in time.

6. Conclusion and Outlook

6.1. Summary of Optimization Results and BMS Implications (HPI and EET)

This study presents a controlled comparison of compression techniques across CNN, TCN, LSTM, and GRU architectures for SOH prediction. The experiments show that the effectiveness of model compression is strongly architecture-dependent. Structured pruning reduces the latency of the convolutional architectures because it directly removes computational operations. In contrast, pruning has almost no effect on the latency of the LSTM and GRU models, since their recurrent layers remained unchanged and continue to dominate execution time. INT8 quantization consistently reduces model storage and is necessary for deploying the recurrent architectures on the target microcontroller.

No single architecture is optimal for every deployment objective. The compressed CNN models provide the most favorable trade-off between model size and prediction accuracy, whereas the GRU achieves the lowest inference latency. Among the uncompressed baselines, the GRU provides the lowest cell-macro MAE and RMSE, while the TCN has the smallest FP32 parameter footprint. Power consumption largely follows the latency trends, indicating that shorter execution time is more important than small variations in power.

For a BMS, optimization is useful only when the reduction in resource demand preserves an SOH estimate that remains sufficiently accurate for monitoring, maintenance planning, and operating-limit adaptation. The combined structured-pruning and INT8-quantization variants reduce parameter storage by approximately 80.0–81.5% for the CNN, GRU, and LSTM, while the absolute change in normalized MAE remains below 0.0008. These results indicate that substantial storage reductions are possible without a corresponding increase of the same magnitude in estimation error. The small accuracy improvements observed for some variants should not be interpreted as a general benefit of compression, but rather as evidence that the optimized models remain within the accuracy range of their baselines.

The TCN result also demonstrates why a BMS implementation cannot be selected from prediction accuracy or a nominal precision label alone. Although the TCN is the smallest FP32 baseline, its current combined optimization path reduces parameter storage by only 29.2%. In particular, quantization provides no further storage reduction after pruning for the evaluated implementation. The achievable benefit, therefore, depends on the architecture, the supported operators, and the representation generated by the deployment toolchain. Relative compression and absolute deployed footprint must consequently be considered together.

From a system perspective, reduced model storage can create flash headroom for firmware, diagnostics, communication functions, and future model updates. However, parameter count and stored model size do not directly determine runtime RAM, latency, or energy consumption. These quantities must be evaluated on the target hardware because conversion overhead and operator support can offset theoretical savings.

Overall, the central contribution of this work is a reproducible, accuracy-cost analysis that connects model optimization directly to embedded design decisions. Rather than recommending one universally optimal model, the results provide a basis for selecting an architecture and compression level according to the application's accuracy, memory, latency, and power constraints.

Figure Directory (Working Draft)

- Figure 1: Study concept from baseline training to compression and embedded evaluation.
- Figure 2: Literature positioning map and research gap framing.

- Figure 3: SOH trajectories with train/validation/test split highlighting.
- Figure 4: Architecture panel for LSTM, GRU, CNN, and TCN.
- Figure 5: Training and baseline-selection workflow.
- Figure 6: Optimization workflow (pruning, quantization, distillation, combined branch).
- Figure 7: Embedded benchmark setup and metric acquisition flow.
- Figure 8: Baseline MAE/RMSE and footprint comparison.
- Figure 9: Pruning sensitivity and pruning Pareto view.
- Figure 10: Quantization sensitivity and precision-resource trade-off.
- Figure 11: Distillation and combined optimization scatter with non-dominated points.
- Figure 12: Cross-architecture decision map.
- Figure 13: Discussion matrix linking deployment constraints to model-method choices.

Data availability (placeholder)

- Dataset DOI, code repository links, and benchmark scripts will be inserted in the final manuscript version.

References

[1] B. Lin, Z. Li, Towards world's low carbon development: The role of clean energy, Applied Energy 307 (2022) 118160. doi:10.1016/j.apenergy.2021.118160.
URL https://linkinghub.elsevier.com/retrieve/pii/S0306261921014331

[2] K. R. Ngoy, V. T. Lukong, K. O. Yoro, J. B. Makambo, N. C. Chukwuati, C. Ibegbulam, O. Eterigho-Ikelegbe, K. Ukoba, T.-C. Jen, Lithium-ion batteries and the future of sustainable energy: A comprehensive review, Renewable and Sustainable Energy Reviews 223 (2025) 115971. doi:10.1016/j.rser.2025.115971.
URL https://linkinghub.elsevier.com/retrieve/pii/S1364032125006446

[3] Y. Chen, Y. Kang, Y. Zhao, L. Wang, J. Liu, Y. Li, Z. Liang, X. He, X. Li, N. Tavajohi, B. Li, A review of lithium-ion battery safety concerns: The issues, strategies, and testing standards, Journal of Energy Chemistry 59 (2021) 83–99. doi:10.1016/j.jechem.2020.10.017.
URL https://linkinghub.elsevier.com/retrieve/pii/S2095495620307075

[4] F. N. U. Khan, M. G. Rasul, A. Sayem, N. Mandal, Maximizing energy density of lithium-ion batteries for electric vehicles: A critical review, Energy Reports 9 (2023) 11–21. doi:10.1016/j.egyr.2023.08.069.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352484723012118

[5] J. Zhang, H. Huang, G. Zhang, Z. Dai, Y. Wen, L. Jiang, Cycle life studies of lithium-ion power batteries for electric vehicles: A review, Journal of Energy Storage 93 (2024) 112231. doi:10.1016/j.est.2024.112231.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352152X24018176

[6] M. U. Ali, A. Zafar, S. H. Nengroo, S. Hussain, M. Junaid Alvi, H.-J. Kim, Towards a Smarter Battery Management System for Electric Vehicle Applications: A Critical Review of Lithium-Ion Battery State of Charge Estimation, Energies 12 (3) (2019) 446. doi:10.3390/en12030446.
URL https://www.mdpi.com/1996-1073/12/3/446

[7] N. Noura, L. Boulon, S. Jemeï, A Review of Battery State of Health Estimation Methods: Hybrid Electric Vehicle Challenges, World Electric Vehicle Journal 11 (4) (2020) 66. doi:10.3390/wevj11040066.
URL https://www.mdpi.com/2032-6653/11/4/66

[8] T. Tao, C. Ji, J. Dai, J. Rao, J. Wang, W. Sun, J. Romagnoli, Data-based health indicator extraction for battery SOH estimation via deep learning, Journal of Energy Storage 78 (2024) 109982. doi:10.1016/j.est.2023.109982.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352152X23033819

[9] S. Jin, X. Sui, X. Huang, S. Wang, R. Teodorescu, D.-I. Stroe, Overview of Machine Learning Methods for Lithium-Ion Battery Remaining Useful Lifetime Prediction, Electronics 10 (24) (2021) 3126. doi:10.3390/electronics10243126.
URL https://www.mdpi.com/2079-9292/10/24/3126

[10] J. Yao, H. Zhao, J. Kowal, Fast-adaptive early-stage remaining useful life prediction of lithium-ion batteries with meta-learning, Journal of Power Sources 660 (2025) 238569. doi:10.1016/j.jpowsour.2025.238569.
URL https://linkinghub.elsevier.com/retrieve/pii/S037877532502405X

[11] M.-K. Tran, M. Mathew, S. Janhunen, S. Panchal, K. Raahemifar, R. Fraser, M. Fowler, A comprehensive equivalent circuit model for lithium-ion batteries, incorporating the effects of state of health, state of charge, and temperature on model parameters, Journal of Energy Storage 43 (2021) 103252. doi:10.1016/j.est.2021.103252.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352152X2100949X

[12] H. Hu, C. Liang, X. Huang, H. Mo, C. Zou, S. Tao, ONET: Operator network for randomized and robust battery health estimation using operation condition and cycling data matching, Journal of Power Sources 672 (2026) 239592. doi:10.1016/j.jpowsour.2026.239592.
URL https://linkinghub.elsevier.com/retrieve/pii/S0378775326003423

[13] P. Iurilli, C. Brivio, R. E. Carrillo, V. Wood, Physics-Based SoH Estimation for Li-Ion Cells, Batteries 8 (11) (2022) 204. doi:10.3390/batteries8110204.
URL https://www.mdpi.com/2313-0105/8/11/204

[14] H. Tu, S. Moura, Y. Wang, H. Fang, Integrating physics-based modeling with machine learning for lithium-ion batteries, Applied Energy 329 (2023) 120289. doi:10.1016/j.apenergy.2022.120289.
URL https://linkinghub.elsevier.com/retrieve/pii/S030626192201546X

[15] C. Chang, S. Wang, C. Tao, J. Jiang, Y. Jiang, L. Wang, An improvement of equivalent circuit model for state of health estimation of lithium-ion batteries based on mid-frequency and low-frequency electrochemical impedance spectroscopy, Measurement 202 (2022) 111795. doi:10.1016/j.measurement.2022.111795.
URL https://linkinghub.elsevier.com/retrieve/pii/S0263224122009976

[16] G. R. Sylvestrin, J. N. Maciel, M. L. M. Amorim, J. P. Carmo, J. A. Afonso, S. F. Lopes, O. H. Ando Junior, State of the Art in Electric Batteries' State-of-Health (SoH) Estimation with Machine Learning: A Review, Energies 18 (3) (2025) 746. doi:10.3390/en18030746.
URL https://www.mdpi.com/1996-1073/18/3/746

[17] Z. Yang, Y. Li, Y. Sun, X. Feng, R. Pan, Y. Zhao, State of Health Estimation of Microgrid Energy Storage Batteries Based on An Improved Temporal Convolutional Network, in: 2024 IEEE PES 16th Asia-Pacific Power and Energy Engineering Conference (APPEEC), IEEE, Nanjing, China, 2024, pp. 1–5. doi:10.1109/APPEEC61255.2024.10922341.
URL https://ieeexplore.ieee.org/document/10922341/

[18] J. Schulze, N. Strassenburg, T. Rabl, PQ Bench: Benchmarking Pruning and Quantization Techniques, in: Proceedings of the Workshop on Data Management for End-to-End Machine Learning, ACM, Berlin Germany, 2025, pp. 1–6. doi:10.1145/3735654.3735940.
URL https://dl.acm.org/doi/10.1145/3735654.3735940

[19] D. P. Pau, A. Aniballi, Tiny Machine Learning Battery State-of-Charge Estimation Hardware Accelerated, Applied Sciences 14 (14) (2024) 6240, read_Status: To Read Read_Status_Date: 2026-08-06T10:26:11.747Z. doi:10.3390/app14146240.
URL https://www.mdpi.com/2076-3417/14/14/6240

[20] F. Rzepka, Design of Experiments for Cyclic Aging of LFP 18650 Cells: Impact of Charge/Discharge C-Rates, Depth of Discharge, and Temperature on Battery Degradation (Nov. 2025). doi:https://doi.org/10.14279/depositonce-24800.

[21] C. Nguyen Van, D. T. Quang, Estimation of SoH and internal resistances of Lithium ion battery based on LSTM network, International Journal of Electrochemical Science 18 (6) (2023) 100166. doi:10.1016/j.ijoes.2023.100166.
URL https://linkinghub.elsevier.com/retrieve/pii/S1452398123001931

[22] G. Xu, J. Xu, Y. Zhu, LSTM-based estimation of lithium-ion battery SOH using data characteristics and spatio-temporal attention, PLOS ONE 19 (12) (2024) e0312856. doi:10.1371/journal.pone.0312856.
URL https://dx.plos.org/10.1371/journal.pone.0312856

[23] L. Ungurean, M. V. Micea, G. Cârstoiu, Online state of health prediction method for lithium-ion batteries, based on gated recurrent unit neural networks, International Journal of Energy Research 44 (8) (2020) 6767–6777, read_Status: To Read Read_Status_Date: 2026-08-05T14:41:17.912Z. doi:10.1002/er.5413.
URL https://onlinelibrary.wiley.com/doi/10.1002/er.5413

[24] D. Vaishnani, R. Vaghela, J. Sarda, A. Thakkar, Y. Nasit, A. Khokhariya, A Deep Learning Approach for State of Health Estimation in Lithium-ion Batteries, in: 2024 International Conference on Modeling, Simulation & Intelligent Computing (MoSICom), IEEE, Dubai, United Arab Emirates, 2024, pp. 385–390. doi:10.1109/MoSICom63082.2024.10880965.
URL https://ieeexplore.ieee.org/document/10880965/

[25] J. Bi, J.-C. Lee, H. Liu, Performance Comparison of Long Short-Term Memory and a Temporal Convolutional Network for State of Health Estimation of a Lithium-Ion Battery using Its Charging Characteristics, Energies 15 (7) (2022) 2448. doi:10.3390/en15072448.
URL https://www.mdpi.com/1996-1073/15/7/2448

[26] A. Aliberti, F. Boni, A. Perol, M. Zampolli, R. J. P. Jaboeuf, P. Tosco, E. Macii, E. Patti, Comparative Analysis of Neural Networks Techniques for Lithium-ion Battery SOH Estimation, in: 2022 IEEE 46th Annual Computers, Software, and Applications Conference (COMPSAC), IEEE, Los Alamitos, CA, USA, 2022, pp. 1355–1361. doi:10.1109/COMPSAC54236.2022.00214.
URL https://ieeexplore.ieee.org/document/9842676/

[27] T. Kim, Y. Seo, S. Barde, Edge-compatible SOH estimation for Li-ion batteries via hybrid knowledge distillation and model compression, Journal of Energy Storage 135 (2025) 118275. doi:10.1016/j.est.2025.118275.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352152X25029883

[28] Product Specification JGCFR18650-1800mAh-3.2V.

[29] D. Kucevic, B. Tepe, S. Englberger, A. Parlikar, M. Mühlbauer, O. Bohlen, A. Jossen, H. Hesse, Standard battery energy storage system profiles: Analysis of various applications for stationary energy storage systems using a holistic simulation framework, Journal of Energy Storage 28 (2020) 101077. doi:10.1016/j.est.2019.101077.
URL https://linkinghub.elsevier.com/retrieve/pii/S2352152X19309016
[30] Gongfan Fang, DepGraph, https://github.com/VainF/Torch-Pruning, accessed: 2026-09-02 (2020).